

# Why Are AI Agent–Involved Pull Requests (Fix-Related) Remain Unmerged? An Empirical Study

Khairul Alam\*

Department of Computer Science  
University of Saskatchewan  
Saskatoon, Saskatchewan, Canada  
kha060@usask.ca

Saikat Mondal

Department of Computer Science  
University of Saskatchewan  
Saskatoon, Saskatchewan, Canada  
saikat.mondal@usask.ca

Banani Roy

Department of Computer Science  
University of Saskatchewan  
Saskatoon, Saskatchewan, Canada  
banani.roy@usask.ca

## Abstract

Autonomous coding agents (e.g., OpenAI Codex, Devin, GitHub Copilot) are increasingly used to generate fix-related pull requests (PRs) in real-world software repositories. However, their practical effectiveness depends on whether project maintainers accept and merge these contributions. In this paper, we present an empirical study of AI agent–involved fix-related PRs, examining both their integration outcomes, latency, and the factors that hinder successful merging. We first analyze 8,106 fix-related PRs authored by five widely used AI coding agents from the AIDEV-POP dataset to quantify the proportions of PRs that are merged, closed without merging, or remain open. We then conduct a manual analysis of a statistically significant sample of 326 closed but unmerged PRs, spending approximately 100 person-hours to construct a structured catalog of 12 failure reasons. Our results indicate that test case failures and prior resolution of the same issues by other PRs are the most common causes of non-integration, whereas build or deployment failures are comparatively rare. Overall, our findings expose key limitations of current AI coding agents in real-world settings and highlight directions for their further improvement and for more effective human-AI collaboration in software maintenance.

## CCS Concepts

• **Information systems** → *Information retrieval*; Retrieval tasks and goals;

## Keywords

AI Agents, Pull Request, Autonomous Coding Agents, Bug Fixing, Human-AI Collaboration

## ACM Reference Format:

Khairul Alam, Saikat Mondal, and Banani Roy. 2026. Why Are AI Agent–Involved Pull Requests (Fix-Related) Remain Unmerged? An Empirical Study. In *23rd International Conference on Mining Software Repositories (MSR '26)*, April 13–14, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3793302.3793611>

## 1 Introduction

Modern autonomous coding agents, such as OpenAI Codex, Devin, Cursor, and GitHub Copilot, are increasingly capable of performing

end-to-end software development tasks, including feature implementation, debugging, bug fixing, documentation, and even code review, with minimal human oversight [4, 10, 20]. These agents are already integrated into thousands of real-world software development projects and often function as virtual teammates that assist developers throughout the software development lifecycle [10]. Their ability to generate PRs, respond to review feedback, and iteratively refine code highlights the growing potential of automated agents to support software engineering workflow [7].

Despite their rapid adoption, the effectiveness of such agents ultimately depends on whether their contributions are *accepted and integrated* into real software systems [22]. In modern development workflows, PRs are the primary mechanism for reviewing, negotiating, and merging changes, making PR acceptance a key indicator of successful human-AI collaboration [21]. Among different contribution types, *fix-related PRs* are particularly critical, as they directly impact software reliability and user trust, and are therefore subject to stricter correctness, testing, and validation requirements.

Prior studies on automated program repair [4, 23] and LLM-based bug fixing [18] demonstrate that, although AI agents can often generate syntactically correct patches, many fixes fail to address root causes, introduce unintended side effects, or lack sufficient test validation [6, 11]. As a result, such fixes frequently fail to pass code review and are ultimately unmerged [1, 8, 13, 17]. For example, in pull request #553 to the Azure/CloudShell<sup>1</sup> repository, an AI agent proposed a fix to resolve a version mismatch among Microsoft.Graph.PowerShell modules by aligning dependency versions so that modules can be imported without assembly errors. Despite multiple automated commits in response to reviewer feedback, the PR repeatedly failed CI tests, did not satisfy reviewer expectations, and was ultimately closed without merging by a maintainer. This case illustrates that generating a plausible patch alone is often insufficient for successful integration when tests or project constraints remain unaddressed. Such observations motivate a systematic investigation into how AI agent-authored fix-related PRs perform in real-world review workflows and why many of them are unmerged.

Existing research on AI-assisted software development primarily emphasizes benchmark-based evaluations and task-level success rates [8, 17], or developer productivity outcomes [5, 14], often abstracting away the downstream PR review and integration process. As a result, there is limited empirical understanding of how AI agent-generated fixes perform within real-world PR workflows under human review and project constraints [2, 16], and, in particular, why fix-related PRs authored by AI agents are closed without being



This work is licensed under a Creative Commons Attribution 4.0 International License. *MSR '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2474-9/26/04

<https://doi.org/10.1145/3793302.3793611>

<sup>1</sup><https://github.com/Azure/CloudShell/pull/553>

merged. Such closed but unmerged PRs represent wasted computational effort and increased review overhead for maintainers, while simultaneously exposing practical limitations in agent reasoning, validation strategies, and interaction with human reviewers.

To address this gap, we conduct an empirical study of fix-related PRs authored by five widely used AI coding agents using the AIDEV-POP dataset [9]. We analyze integration outcomes at scale and construct a structured catalog of failure reasons through in-depth manual analysis of closed but unmerged PRs. In particular, we investigate the following research questions:

**RQ1 (Outcome and Latency Analysis): What proportion of AI agent-involved fix-related PRs are merged, closed without merging, or remain open? What is the submission-to-merge time for merged PRs?** Autonomous coding agents increasingly submit PRs to real-world repositories, but their effectiveness depends on whether these contributions are successfully integrated. Merge outcomes provide a concrete signal of integration, reflecting correctness, test compliance, and alignment with project conventions, while submission-to-merge time captures additional integration effort such as review and coordination overhead. However, there is limited empirical evidence on how often AI agent-authored fix-related PRs are merged, closed without merging, or remain open, or how long successful merges take. Accordingly, this RQ evaluates agent effectiveness by analyzing both integration outcomes and submission-to-merge time of fix-related PRs in real-world workflows.

**RQ2 (Failure Analysis): Why are AI agent-involved fix-related PRs closed without merging?** PRs that are closed without merging represent concrete failure points in human-AI collaboration, where AI agent-generated contributions do not meet project or maintainer expectations. Analyzing such unmerged PRs enables the construction of a structured catalog of failure causes that explain why AI-generated fixes fail to integrate in practice. Understanding these failure causes is essential for diagnosing the practical limitations of current AI coding agents and informing improvements to agent design, PR review, and integration workflows.

By jointly analyzing integration outcomes and failure causes of AI agent-authored fix-related PRs, our study clarifies where and why such contributions fail in practice, informing both agent improvement and more effective human-AI collaboration.

The **replication package** is available in our online appendix [15].

## 2 Methodology

We adopt a mixed-methods approach, combining quantitative analysis of PR integration outcomes with qualitative analysis of closed but unmerged PRs to construct a catalog of failure reasons.

### 2.1 Dataset Selection

We use the AIDEV-POP dataset [9] released in November 2025, a large-scale collection of AI-authored PRs mined from GitHub repositories. The dataset contains 33,596 PRs, of which 8,106 are classified as fix-related and authored by five widely used AI coding agents: OpenAI Codex (4,338), GitHub Copilot (1,993), Devin (1,249), Cursor (411), and Claude Code (115). We focus exclusively

on fix-related PRs because they directly address defects or incorrect behavior and consequently undergo more rigorous review, testing, and validation processes. This makes them particularly suitable for evaluating the practical effectiveness of AI agents in real-world collaborative software development.

### 2.2 Analysis of PR Integration Outcomes (RQ1)

To assess the integration outcomes of AI agent-authored fix-related PRs, each PR is classified into one of three outcome states: *merged*, *closed without merging*, or *open* based on its status at the time of data collection. We report both the *overall* outcome distribution and the *agent-wise* distributions to examine differences in integration outcomes across AI coding agents.

For merged PRs, we analyze *time-to-merge* to characterize integration latency. Specifically, we compute time-to-merge as the difference (in hours) between the PR creation (*created\_at*) and closing timestamp (*closed\_at*) recorded by GitHub. We then analyze the distribution of time-to-merge and compare integration latency across different AI coding agents to examine agent-level differences in how quickly fix-related PRs are merged in practice.

### 2.3 Analysis of Failure Causes in Closed but Unmerged PRs (RQ2)

To address RQ2, we focus on fix-related PRs that were closed without merging. From the 2,113 unmerged cases identified in RQ1, we randomly sample 326 PRs, which provides a 95% confidence level with a 5% margin of error [12, 19]. This sample size is sufficient for statistically representative qualitative analysis.

We then conduct a manual qualitative analysis using an open coding approach. Two authors independently label each PR, spending approximately *100 person-hours* in total, following an initial calibration phase, in which they jointly analyzed 20 PRs to establish a shared understanding of the coding procedure and criteria. The calibration PRs are excluded from the final sample of 326 PRs. For each PR, the annotators examine the PR description, code changes, review comments (when available), commit history, and closure context to identify the primary reason for the unsuccessful merge outcome. Failure categories are derived inductively from the data rather than predefined.

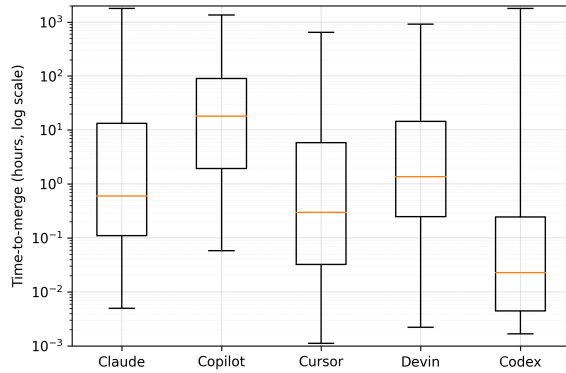
To assess coding reliability, we compute Cohen's kappa [3], obtaining a value of 0.82, indicating substantial inter-rater agreement. All disagreements are resolved through discussion to reach a consensus. Through this process, we construct a structured catalog of failure causes that explain why AI agent-authored fix-related PRs are closed without being merged.

## 3 Study Findings

This section presents the empirical findings of our study.

### 3.1 Findings of RQ1: Integration Outcomes of Fix-Related PRs

Table 1 summarizes the integration outcomes of fix-related PRs authored by different AI coding agents. Overall, a majority of fix-related PRs are successfully merged (65.0%), indicating that AI agents can frequently produce acceptable fixes in real-world



**Figure 1: Distribution of time-to-merge for merged fix-related PRs across AI coding agents (log scale).**

projects. However, the outcomes vary substantially across agents. OpenAI Codex exhibits a notably high merge rate (81.6%), whereas GitHub Copilot and Devin show much lower merge rates (42.4% and 42.9%, respectively) and a higher prevalence of closed-but-unmerged PRs. In particular, more than half of Devin’s fix-related PRs are closed without merging, highlighting significant integration challenges. These differences suggest that while AI-generated fixes are often viable, their likelihood of successful integration depends strongly on the agent and on how its contributions align with project expectations and review workflows.

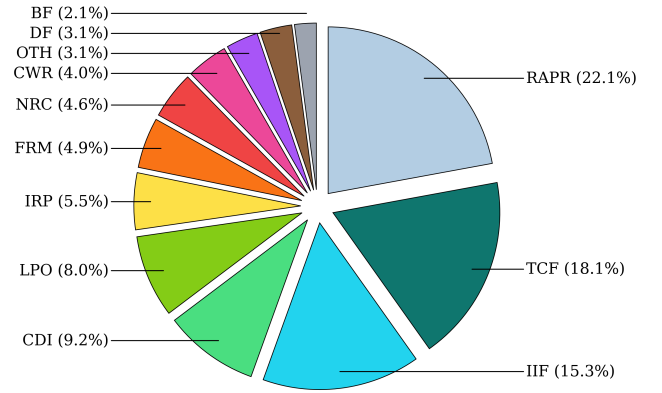
**Table 1: Integration outcomes of fix-related PRs authored by AI coding agents**

| AI Agent       | Total PRs    | Merged               | Closed w/o Merge     | Open              |
|----------------|--------------|----------------------|----------------------|-------------------|
| OpenAI Codex   | 4,338        | 3,539 (81.6%)        | 616 (14.2%)          | 183 (4.2%)        |
| GitHub Copilot | 1,993        | 845 (42.4%)          | 713 (35.8%)          | 435 (21.8%)       |
| Devin          | 1,249        | 536 (42.9%)          | 675 (54.0%)          | 38 (3.0%)         |
| Cursor         | 411          | 281 (68.4%)          | 84 (20.4%)           | 46 (11.2%)        |
| Claude Code    | 115          | 66 (57.4%)           | 25 (21.7%)           | 24 (20.9%)        |
| <b>Total</b>   | <b>8,106</b> | <b>5,267 (65.0%)</b> | <b>2,113 (26.1%)</b> | <b>726 (8.9%)</b> |

We further analyze the time required for AI agent–authored fix-related PRs to be successfully merged. Fig. 1 shows the distribution of merge latency across different AI agents. The results indicate substantial variability both within and across agents, with most merged PRs integrated quickly, while a smaller fraction experiences prolonged delays. In particular, OpenAI Codex exhibits consistently shorter merge times with a tightly concentrated interquartile range, whereas Copilot and Devin show broader distributions and heavier upper tails, reflecting greater variability in integration latency. Overall, the figure highlights that while many AI-generated fixes are merged rapidly, merge latency is strongly influenced by the agent and the complexity of the review process.

### 3.2 Findings of RQ2: Failure Causes of Closed but Unmerged PRs

Our manual analysis of closed-but-unmerged fix-related PRs resulted in a structured catalog of **12 distinct failure reasons**. Figure 2 illustrates the distribution of these reasons across the analyzed 326 PRs. Overall, the findings indicate that unsuccessful integration arises from a combination of *technical shortcomings* and *process-related factors*. We describe each failure reason below.



**Figure 2: Distribution of failure causes (R1–R12) for closed but unmerged fix-related PRs.**

**(R1) Resolved by Another Pull Request (RAPR).** This is the most frequent cause, accounting for 22.1% (72 PRs). In these cases, the reported issue was resolved by a different PR that was merged earlier or in parallel. We identified such cases by tracing PR discussions and linked issues and verifying that the issue was closed due to another merged PR. For example, the PR “*Fix JsonValue serialization InvalidCastException in Orleans JsonCodec*” was closed after the issue was resolved by a separate merged PR, “*Improve support for serializing System.Text.Json types*.”

**(R2) Test Case Failures (TCF).** 18.1% (59 PRs) were closed because the proposed changes failed existing tests or introduced new test failures. Reviewers explicitly mentioned failing tests as the primary reason for non-integration, such as in the PR “*Fix debug expression hover for TypeScript non-null assertions (!)*.”

**(R3) Incorrect or Incomplete Fixes (IIF).** We find 15.3% (50 PRs) in which the proposed fixes were incorrect, incomplete, or failed to resolve the reported issue. Reviewers indicated that the changes did not constitute a valid fix or introduced incorrect behavior. An example is “*Fix division by zero handling inconsistency in arithmetic optimization*”, which was closed after reviewers determined the fix was invalid.

**(R4) Fix Rejected by Maintainers (FRM).** In 4.9% (16 PRs), fixes underwent review and revision but were ultimately rejected by maintainers. Although AI agents addressed the reviewer feedback, maintainers decided not to accept the final solutions. For instance, “*Fix task restart to re-read tasks.json configuration*” was closed after multiple review iterations.

**(R5) Deployment Failures (DF).** 3.1% (10 PRs) were closed due to deployment or runtime failures encountered during integration or release. Maintainers reported that the changes could not be deployed, as in “*Fix smithery build config and bundle OpenAPI spec*.”

**(R6) Build Failures (BF).** We identify 2.1% (7 PRs) where proposed changes triggered build failures, typically reflected by failing CI checks. Maintainers explicitly cited build errors as the reason for closure, for example in “*Fix fs.writeSync options parsing*.”

**(R7) Low Priority or Obsolete Issue (LPO).** In 8.0% (26 PRs), the reported issues were deprioritized or rendered obsolete, making the fixes unnecessary. For example, “*Fix DataProduct assets persistence issue by removing premature sorting*” was closed as subsequent codebase changes resolved the issue.

**Table 2: Agent-wise distribution of failure causes for closed-but-unmerged fix-related PRs.**

| Failure Cause                | Claude    | Copilot    | Cursor    | Devin      | Codex      |
|------------------------------|-----------|------------|-----------|------------|------------|
| Resolved by Another PR       | 5 (27.8%) | 47 (31.8%) | 5 (26.3%) | 7 (14.0%)  | 8 (8.8%)   |
| Test Case Failures           | 0 (0.0%)  | 6 (4.1%)   | 0 (0.0%)  | 3 (6.0%)   | 50 (54.9%) |
| Incorrect / Incomplete Fixes | 0 (0.0%)  | 30 (20.3%) | 0 (0.0%)  | 4 (8.0%)   | 13 (14.3%) |
| Closed Due to Inactivity     | 1 (5.6%)  | 2 (1.4%)   | 0 (0.0%)  | 27 (54.0%) | 0 (0.0%)   |
| Low Priority / Obsolete      | 4 (22.2%) | 8 (5.4%)   | 0 (0.0%)  | 0 (0.0%)   | 4 (4.4%)   |
| Incomplete Review            | 0 (0.0%)  | 18 (12.2%) | 0 (0.0%)  | 0 (0.0%)   | 4 (4.4%)   |
| Fix Rejected                 | 0 (0.0%)  | 13 (8.8%)  | 0 (0.0%)  | 4 (8.0%)   | 13 (14.3%) |
| No Review Conducted          | 1 (5.6%)  | 7 (4.7%)   | 1 (5.3%)  | 2 (4.0%)   | 4 (4.4%)   |
| Closed w/o Rationale         | 3 (16.7%) | 1 (0.7%)   | 8 (42.1%) | 1 (2.0%)   | 0 (0.0%)   |
| Deployment Failures          | 0 (0.0%)  | 0 (0.0%)   | 2 (10.5%) | 3 (6.0%)   | 5 (5.5%)   |
| Build Failures               | 1 (5.6%)  | 1 (0.7%)   | 0 (0.0%)  | 1 (2.0%)   | 4 (4.4%)   |
| Other Reasons                | 1 (5.6%)  | 5 (3.4%)   | 1 (5.3%)  | 0 (0.0%)   | 3 (3.3%)   |
| <b>Total PRs</b>             | <b>18</b> | <b>148</b> | <b>19</b> | <b>50</b>  | <b>91</b>  |

**(R8) Incomplete Review Process (IRP).** 5.5% (18 PRs) experienced partial review activity that did not reach a conclusion. Reviewers provided initial feedback, but the review process was not completed, and the PRs were eventually closed, such as in “*Fix duplicate and unfitting slugs in resourceDefinition.json.*”

**(R9) Closed Due to Inactivity (CDI).** We find 9.2% (30 PRs) that were automatically closed due to prolonged inactivity. These PRs showed no further updates within the project-defined time window and were closed by repository automation.

**(R10) Closed Without Explicit Rationale (CWR).** In 4.0% (13 PRs), PRs were closed without any comments or explanation, leaving the closure rationale unclear. For example, “*Fix module resolution for observability*” was closed unmerged without explanation.

**(R11) No Review Conducted (NRC).** 4.6% (15 PRs) were closed without any reviewer engagement, even though AI agents explicitly requested review. These cases reflect a lack of review participation rather than technical evaluation.

**(R12) Other Reasons (OTH).** The remaining 3.1% (10 PRs) were closed for miscellaneous reasons, including repository archival, infrastructure or authentication issues, lack of substantive code changes, or closure to submit a cleaner or revised PR.

We further analyze agent-wise failure causes by normalizing each category by the total number of closed-but-unmerged PRs per agent (Table 2). For OpenAI Codex, failures are dominated by test case failures (50 out of 91, 54.9%), indicating validation-related issues rather than review or prioritization. In contrast, Devin is primarily affected by inactivity-related closures (27 out of 50, 54.9%), reflecting challenges sustaining engagement during longer review cycles. GitHub Copilot shows several notable failure profiles, with issues resolved by other PRs as the largest category (47 out of 148, 31.7%), suggesting redundancy and timing effects alongside technical shortcomings. For Cursor and Claude Code, failures are relatively few and skew toward prioritization- or redundancy-related causes rather than systematic technical errors.

**Synthesis of Failure Causes.** To better understand the nature of unsuccessful integration, we group the identified failure reasons into three high-level categories: *technical*, *process-related*, and *priority-related* causes. **Technical causes** include cases where the proposed changes were incorrect or incomplete (IIF), failed test cases (TCF), or triggered build or deployment failures (BF, DF). These failures indicate limitations in the agents’ ability to generate fully correct and validation-ready fixes. **Process-related causes**

capture breakdowns in the pull-request workflow, including fixes rejected after review (FRM), incomplete review processes (IRP), absence of review (NRC), closures without explicit rationale (CWR), and closures due to inactivity (CDI). Such cases highlight challenges in review interaction, feedback handling, and sustained engagement. Finally, **priority-related causes** include issues resolved by another PR (RAPR) or deemed low priority or obsolete (LPO), reflecting redundancy, timing mismatches, or shifting maintainer priorities rather than intrinsic technical flaws. Together, this synthesis shows that while many failures stem from technical shortcomings, a substantial portion arises from workflow and prioritization dynamics inherent to real-world collaborative software development.

## 4 Key Findings

**Integration reflects encouraging outcomes with notable limitations.** Across 8,106 fix-related PRs, 65.0% are merged, while 26.1% are closed without merging and 8.9% remain open, indicating that a substantial fraction of AI agent-authored fixes remain unmerged. **Integration success varies notably across agents.** Merge rates differ considerably by agent, ranging from 81.6% (OpenAI Codex) to 42.9% (Devin), suggesting that real-world effectiveness may depend on agent design and alignment with project workflows.

**Validation-related issues are more prevalent than build failures.** Among closed-but-unmerged PRs, test case failures (18.1%) and incorrect or incomplete fixes (15.3%) occur substantially more often than build (2.1%) or deployment failures (3.1%), pointing to limitations in validation readiness rather than basic compilability. **Workflow and timing factors play a non-trivial role.** A notable share of unmerged PRs are closed because the issue was resolved by another PR (22.1%) or due to inactivity (9.2%), indicating that process-related dynamics and timing effects can affect integration outcomes alongside technical quality.

**Merge latency varies substantially across agents.** For merged PRs, submission-to-merge times show considerable variability both within and across agents, suggesting differences in review effort and coordination overhead during integration.

## 5 Conclusion and Future Work

This paper presents an empirical study of AI agent-involved fix-related PRs using the AIDEV-POP dataset. Analyzing 8,106 PRs authored by five widely used AI coding agents, we find that while many AI-generated fixes are merged, a substantial fraction are closed without merging or remain open. Integration success varies across agents, and non-merge outcomes are frequently associated with validation-related issues (e.g., failing tests or incomplete fixes) as well as workflow- and timing-related factors. A qualitative analysis of closed-but-unmerged PRs further yields a structured catalog of 12 failure causes, suggesting that syntactically plausible fixes alone are often insufficient for successful integration. As future work, we plan to investigate test- and context-aware agent designs that better support automated validation, review interactions, and alignment with project-specific expectations, and to examine maintainer-agent interactions over extended review cycles.

## Acknowledgments

This research is supported in part by the Natural Sciences and Engineering Research Council of Canada (NSERC), and by the industry-stream NSERC CREATE in Software Analytics Research (SOAR).

## References

- [1] Khairul Alam and Banani Roy. 2025. What Drives Issue Resolution Speed? An Empirical Study of Scientific Workflow Systems on GitHub. *arXiv preprint arXiv:2512.18852* (2025).
- [2] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 712–721.
- [3] Nicole J-M Blackman and John J Koval. 2000. Interval estimation for Cohen's kappa as a measure of agreement. *Statistics in medicine* 19, 5 (2000), 723–741.
- [4] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. Repairagent: An autonomous, llm-based agent for program repair. *arXiv preprint arXiv:2403.17134* (2024).
- [5] Thomas Dohmke. 2023. The economic impact of the AI-powered developer lifecycle and lessons from GitHub Copilot. *GitHub Blog* (2023).
- [6] John Grundy, Guoliang Ding, and John Hosking. 2005. Deployed software component testing using dynamic validation agents. *Journal of Systems and Software* 74, 1 (2005), 5–14.
- [7] Ahmed E Hassan, Gustavo A Oliva, Dayi Lin, Boyuan Chen, Zhen Ming, et al. 2024. Towards AI-native software engineering (SE 3.0): A vision and a challenge roadmap. *arXiv preprint arXiv:2410.06107* (2024).
- [8] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* (2023).
- [9] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. *AIDev: Studying AI Coding Agents on GitHub*. doi:10.5281/zenodo.16919272
- [10] Hao Li, Haoxiang Zhang, and Ahmed E Hassan. 2025. The rise of ai teammates in software engineering (se) 3.0: How autonomous coding agents are reshaping software engineering. *arXiv preprint arXiv:2507.15003* (2025).
- [11] Yizhou Liu, Pengfei Gao, Xinchun Wang, Jie Liu, Yexuan Shi, Zhao Zhang, and Chao Peng. 2024. Marscode agent: Ai-native automated bug fixing. *arXiv preprint arXiv:2409.00899* (2024).
- [12] Saikat Mondal, Gias Uddin, and Chanchal Roy. 2023. Automatic prediction of rejected edits in stack overflow. *Empirical Software Engineering* 28, 1 (2023), 9.
- [13] Martin Monperrus. 2014. A critical review of "automatic patch generation learned from human-written patches": Essay on the problem statement and the evaluation of automatic software repair. In *Proceedings of the 36th International Conference on Software Engineering*. 234–242.
- [14] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. 2023. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590* (2023).
- [15] Annoymous Random. 2025. *Artifact of the paper "Why Are AI Agent–Involved Pull Requests (Fix-Related) Remain Unmerged? An Empirical Study"*. doi:10.5281/zenodo.18105922
- [16] Peter Rigby, Brendan Cleary, Frederic Painchaud, Margaret-Anne Storey, and Daniel German. 2012. Contemporary peer review in action: Lessons from open source development. *IEEE software* 29, 6 (2012), 56–61.
- [17] Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. 2023. An analysis of the automatic bug fixing performance of chatgpt. In *2023 IEEE/ACM International Workshop on Automated Program Repair (APR)*. IEEE, 23–30.
- [18] Xiaolong Tian. 2024. Evaluating the repair ability of LLM under different prompt settings. In *2024 IEEE International Conference on Software Services Engineering (SSE)*. IEEE, 313–322.
- [19] Shaowei Wang, Tse-Hsun (Peter) Chen, and Ahmed E. Hassan. 2018. How Do Users Revise Answers on Technical Q&A Websites? A Case Study on Stack Overflow. *IEEE Transactions in Software Engineering* (2018), 19.
- [20] Yanlin Wang, Wanjun Zhong, Yanxian Huang, Ensheng Shi, Min Yang, Jiachi Chen, Hui Li, Yuchi Ma, Qianxiang Wang, and Zibin Zheng. 2025. Agents in software engineering: Survey, landscape, and vision. *Automated Software Engineering* 32, 2 (2025), 1–36.
- [21] Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E. Hassan. 2025. On the use of agentic coding: An empirical study of pull requests on github. *arXiv preprint arXiv:2509.14745* (2025).
- [22] Ilya Zakharov, Ekaterina Koshchenko, and Agnia Sergeyuk. 2025. Ai in software engineering: Perceived roles and their impact on adoption. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 1305–1309.
- [23] Quanjun Zhang, Chunrong Fang, Yang Xie, YuXiang Ma, Weisong Sun, Yun Yang, and Zhenyu Chen. 2024. A systematic literature review on large language models for automated program repair. *arXiv preprint arXiv:2405.01466* (2024).